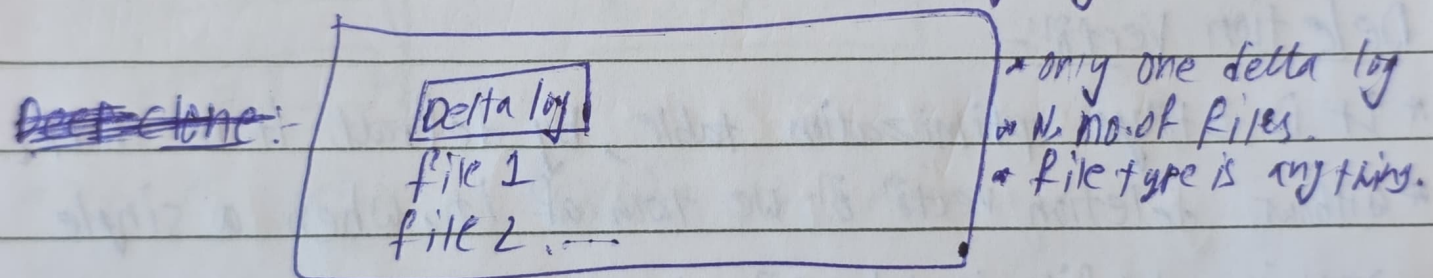


Delta Lake ^{DATE} Data Warehouse

- * It is an open source storage layer on top of data lake
- * Features:-
 1. ACID properties
 2. Time travel
 3. Schema enforcement.
- * It is an open table format

Delta Log:-

- * It is a transaction log
- * Also, in this log ACID properties are going to perform.



Deep clone:-

- * Clone the entire data like Delta log & files
- * It is independent, if any changes happen to original table it is not going to effect.

Ex:-

Create table ^{new} table - name

Deep clone

Old - table - name

Shallow clone:-

- * It clones the Delta log only not the files
- * It is dependent if any changes happen to original table this table also going to be effected.
- * It just points to the original data.

Ex:-

create table new-table-name

Shallow clone

old-table-name.

Deletion Vectors:-

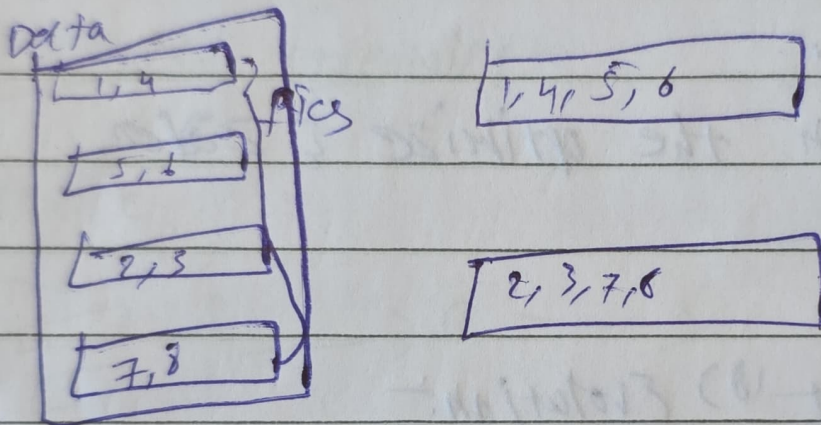
- * It is storage optimization table, by default it is enabled
- * Without deletion vector if we turn off it, when a single row in a datafile is deleted or updated, the entire parquet file must be rewritten.
- * With deletion vector enabled, some Delta operations use deletion vectors to make existing rows as removed without rewriting the parquet file.
- * Subsequent reads on the table resolve current table state by applying the deletions noted by deletion vectors to the most recent table version.

Time travel & version:-

Describe history table_name;

* To find versions

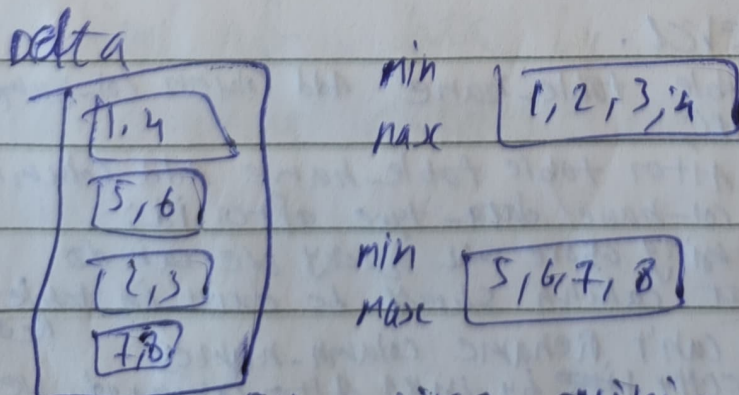
Restore table_name to version as of (number);

Optimization techniques:-1. Optimize:-

* It is going to collise the files

Ex:-

Optimize table_name;

2. Zorden:- ex → Optimize table_name zorden by id;

* main difference is
It will do '
data skipping

* It is also same like optimize, and also includes sorting in data.

3. Liquid clustering:-

- * It divides a group by based on the same category.
- * Clusters are created by dynamically.

ex:-

Create table table-name()

Using delta

Location ' '

Cluster by (id):

- * It is ^{50%} faster than the optimize & Zorder.

Schema Enforcement (b) Evolution:-

- * By using "mergeSchema" Command we can do
- * While writing into External files we have to use
• option ("mergeSchema", true)

Explicit Schema updates:-

- * It is done in log level.

* Add a Column $\rightarrow$ Alter table table-name Add Column col-name data-type;

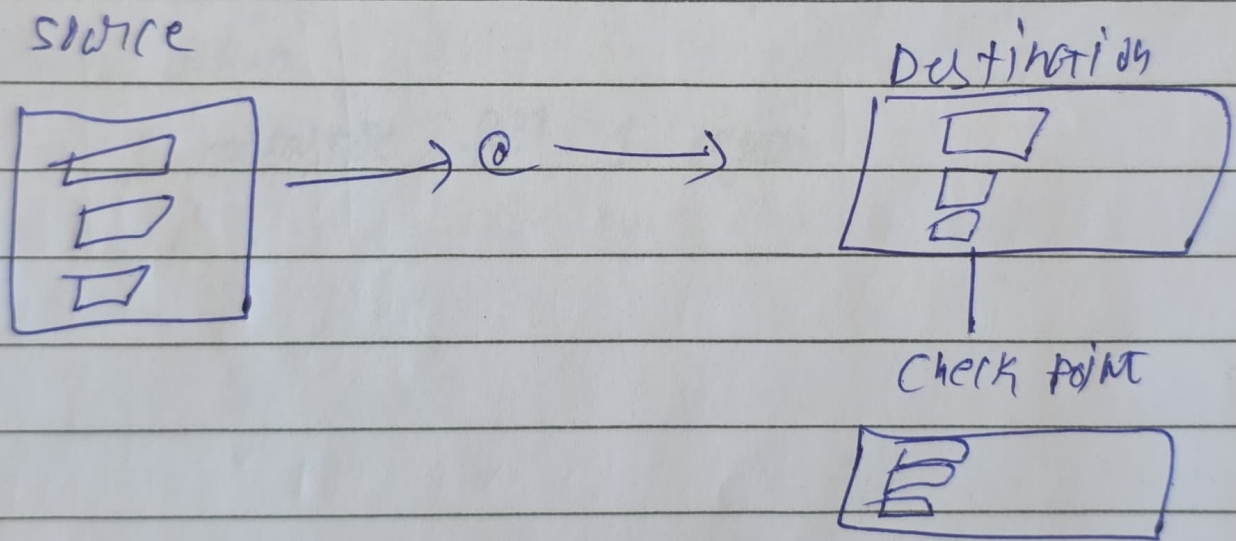
* Add a Column After $\rightarrow$ Alter table table-name Add Column col-name data-type after id;

* Reordering Column $\rightarrow$ By using above sql query we can do that column should be exist in table to Reorder

* Renaming Columns $\rightarrow$ we can't Rename column-names directly ~~the~~ by using Alter Command. We have to enable some-thing in table properties to Rename Column by using alter because we are using Delta Lake.

After doing those thing we have to do changes in files also so that only we can able to perform debug or back tracking, to do that we have to execute "Reorg" Command to change details in files which are present in metadata or log.
 ex:- Reorg table table-name Apply (PURGE);

Structured streaming:-

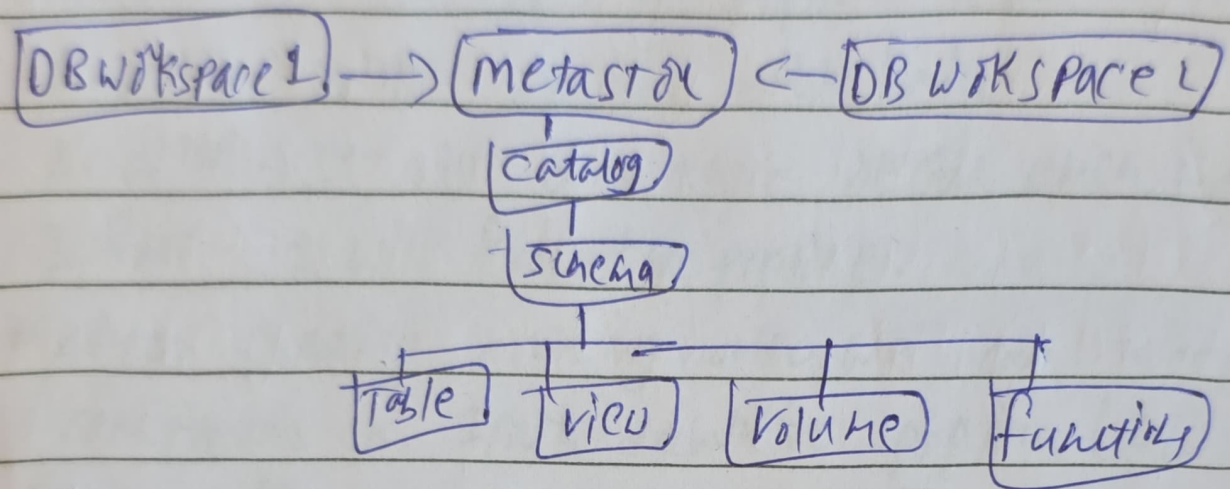


- Delta lake automatically detects the new arrival of data by using check point.
- Check point contains -> commits, stages, metadata

Unity catalog

DATE :/...../.....

* It is a unified data governance solution for data lake



* By connecting different DB workspaces to that metastore those two DB workspaces can communicate each other.

* 1 unity metastore for 1 region.

Data quality

DATE :/...../.....

* It will be create in pipeline level

* There are 3 types of pipeline level

1. Warn $\rightarrow$ warned records also loaded

2. drop $\rightarrow$ It will drop those records which is giving ^{warn}

3. fail $\rightarrow$ It will fail the pipeline.

* After creating everything also we can keep constraints in Spark declarative pipelines (DLT)

ex: ~~@dlt.expect (id is not null)~~

@dlt.expect^{fail} - ~~warn~~ (name, 'col is not null')

@dlt.expect () $\leftarrow$ Default warn.

@dlt.expect^{fail} - drop

@dlt.expect - all - ^{fail} - drop (name)

name = {

'amount' : 'amount > 50',

'age' : 'age > -1 & age < 150'

}

How to give permissions to group:-

DATE:/...../.....

1. Grant use catalog on catalog catalog-name to 'group-name'
2. Grant use schema on schema catalog.schema-name to 'group-name'
3. Grant select on table table-name to 'group-name'

for verify:-

Show grants on table table-name;

How to set row filter to a table:-

Ex: create:-

```
create or replace function catalog.schema.region  
(region string) return region = 'North';
```

Apply ex:-

Alter table table-name set row filter ~~region~~
function-name on (col-name);

To remove row filter:-

Alter table table-name drop row filter;

How to set column filter to a function or Data masking:

Create functions

Create or replace function function-name (email string)
return case

When current-user() = 'user-id' then email
else 'xxx @.gmail.com'

end;

Apply it to table:

Alter table table-name Alter column email set mask
fun-name;

To remove:-

Alter table table-name Alter column email drop
mask;

Comments

create table table_name (col 1, col 2,
 col 3 comment ' ' Column level
 col 4 comment ' ')
 comment ' ' Table level

Tags

set tag on table table_name key = value;
 for table

Alter table table_name Alter Column column-name
 set tags (key = value);
 for Column level.

Verify tags for table-level:- catalog-name
 select * from automotive-catalog-se . information-schem
 table-tags where table-name = 'table_name';

Verify tags for Column-level:-
 select * from catalog-name . ~~schema~~ information-schema.
 Column-tags where table-name = 'table_name';

How to change retention period for table:-

* For this we have to change 2 table properties

1. 'delta.logRetentionDuration' ~~30 days~~
2. 'delta.deletedFileRetentionDuration'

Ex:-

Alter table table-name

SET TBLPROPERTIES

'delta.logRetentionDuration' = 'interval 14 days',
'delta.deletedFileRetentionDuration' = 'interval 14 days';

Vacuum

* Default 7 days

Ex:-

Vacuum table-name; ← To remove files

Vacuum table-name dry run; ← It shows what are the files are going to be deleted

* Preferable run this command first & validate the execute next command.

Describe history table-name; → To get versions

Predictive optimization

* Vacuum & optimize runs automatically without manual scheduling.

Ex:-

```
Alter schema schema-name ENABLE PREDICTIVE  
OPTIMIZATION;
```

Describe history table-name;